GIT HUB

Page :
Date : 7/1/2026

What is Git ?

⇒ Git is a distributed version control system used to track changes in source code and collaborate with others during software development. It helps teams manage changes to code over time, allowing multiple people to work on a project simultaneously without overwriting each other's work.

⇒ Git was developed in 2005 by Linus Torvalds, the creator of the Linux operating system Kernal.

Key Features of Git :-

① Version Control → Keeps a history of changes made to files, so you can review or revert to previous versions.

② Branching and Merging → Enables developers to work on new features or fixes in isolated branches then merge them back into the main codebase.

③ Distributed :- Every developer has a full copy of the repository, including the complete history of changes.

Page :
Date : / /

④ Collaboration : Works well with platforms like GitHub, GitLab, or Bitbucket for remote collaboration and code sharing.

After installing git for the first time :

Git Global Configuration ⇒
- Applies to all repositories for the current user on the computer.
- Does not affect other users or the system as a whole.
- Overrides system config, but can be overridden by local (repo) ~~calling~~ config.

⇒ Where global config is stored.
Path → C:\Users\<username>\.gitconfig

① git config --global user.name "Your Name"
② git config --global user.email "you@example.com"
③ git config --global core.editor "code --wait"
④ git config --global core.autocrlf "input"

⑤ for editing in global Configuration
git config --global -e

⑥ List of all global settings :
git config --global --list

Page :
Date : / /

Git file lifecycle (Stages) :-

- U (Untracked) : New files that Git is not watching yet.
- A (Added / Staged) : Files moved to staging area, ready to be saved.
- C (committed) : Files permanently saved in the project history.

Commands :-

- git status -s : Displays a short, one-line status of your staged and unstaged files.
- git log --oneline : Shows a condensed history of your "saved points" (Commits)

① git init - Initializes a new Git repository by creating a .git directory.
② git status - Shows the current state of the working directory and staging area.
③ git log - Shows the full commit history with detailed metadata.
④ git log --oneline --graph - Shows a visual ASCII graph of commit and branch history.
⑤ git add <file-name> - Stages a specific file for the next commit.

Page :
Date : / /

⑥ git log --online - Displays commit history in a single-line, compact format.

⑦ git add . - stages all chages in the current directory and subdirectories.

⑧ git commit -m "message" - Creates a commit from staged changes with a message

• .gitignore file ⇒ Specifies filed and directories Git should ignore when tracking.

⑨ git reset --soft - Moves HEAD while keeping staged and working directory changes.

⑩ git reset --mixed - Moves HEAD and discards unstaged and keeping working directory. ~~changes~~.

⑪ git reset --hard - Moves HEAD and discards staged and working directory changes.

⑫ git reset --hard HEAD~2 - Resets the branch two commits back and deletes those commits.

* Conflicts ⇒ Occurs when Git cannot automatically merge changes and needs manual resolution.

⑬ git branch <any-name> - Creates a new branch pointing to the current commit.

Page :
Date : / /

(14) git switch <branch-name> - Switches to an existing branch and updates the working directory to match it.

(15) git stash - Temporarily saves uncommitted changes and cleans the working directory.

(16) git stash apply - Restores the most recent stash without removing it.

(17) git stash clear - Permanently deletes all stashes.

(18) git pull - Fetches and merges remote changes into the current branch

Flow for colloborating on a Git Repository

(1) git clone <repo-url> - copies the remote repository to your local machine.

(2) cd repo-name - Moves into the project directory.

(3) git status - Checks the current state of the working directory.

(4) git fetch - Updates remote references without changing local files.

(5) git switch -c feature/your-feature-name - Creates and switches to a new feature branch.

(6) End files - Make your code changes locally.

Page :
Date : / /

⑦ git status -s - Verifies which files were modified.

⑧ git add . - Stages all intended changes

⑨ git commit -m "Describe your changes clearly"
- Commits the changes locally

⑩ git pull origin main - Updates your branch with the latest main branch changes.

⑪ Resolve conflicts if any -- Fix conflicts then git add . and git commit.

⑫ git push -u origin feature/your-feature-name
- Pushes your branch to the remote repository.

⑬ Create Pull Request - Done on the hosting platform (GitHub)

⑭ Review & merge - Team reviews and merges into main.

⑮ git switch main - Switches back to the main branch.

⑯ git pull - Syncs local main with the latest remote changes.

⑰ git branch -d feature/your-feature-name
- Deletes the local feature branch after merge.